



Universidad Autónoma del Carmen

Facultad de Ciencias de la Información

MATERIA

Reconocimiento de patrones

MAESTRO

Jesús Alejandro Flores Sánchez

ALUMNO

Miguel Hernández Méndez

ACTIVIDAD

Tarea Mayo

Ciudad del Carmen, Campeche, 2 de mayo de 2025.

Actividad 6

Conjunto de datos generado

Conjunto de datos clase 1

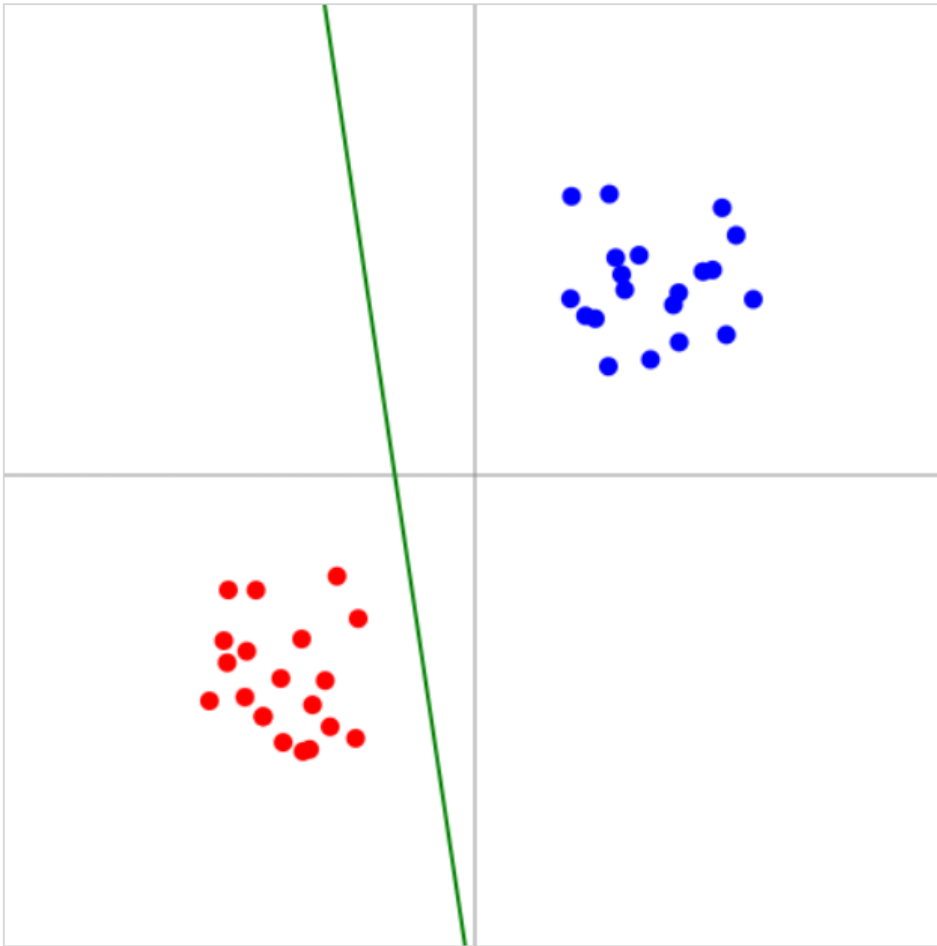
(2.50, 2.86, 1)
(2.65, 1.70, 1)
(0.92, 2.62, 1)
(3.12, 1.76, 1)
(2.26, 1.78, 1)
(2.94, 2.83, 1)
(2.48, 2.74, 1)
(2.00, 2.35, 1)
(1.83, 1.71, 1)
(2.77, 2.36, 1)
(2.43, 2.45, 1)
(1.98, 1.92, 1)
(2.22, 2.17, 1)
(2.89, 2.30, 1)
(1.97, 3.10, 1)
(2.73, 1.85, 1)
(1.88, 2.67, 1)
(2.13, 1.65, 1)
(1.72, 2.47, 1)
(2.59, 1.96, 1)

Conjunto de datos clase -1

(-2.88, -2.67, -1)
(-1.79, -2.56, -1)
(-1.93, -2.80, -1)
(-2.15, -1.62, -1)
(-1.64, -2.45, -1)
(-2.60, -1.73, -1)
(-2.37, -2.58, -1)
(-2.71, -2.22, -1)
(-1.87, -1.93, -1)
(-2.29, -2.18, -1)
(-2.45, -1.98, -1)
(-1.76, -2.87, -1)
(-2.15, -2.11, -1)
(-1.93, -2.32, -1)
(-2.05, -1.75, -1)
(-2.68, -2.01, -1)
(-2.47, -1.68, -1)
(-2.12, -2.50, -1)
(-2.25, -1.82, -1)
(-2.33, -2.40, -1)

Aquí esta modificado el código del perceptrón para que separe el conjunto de datos que generó.

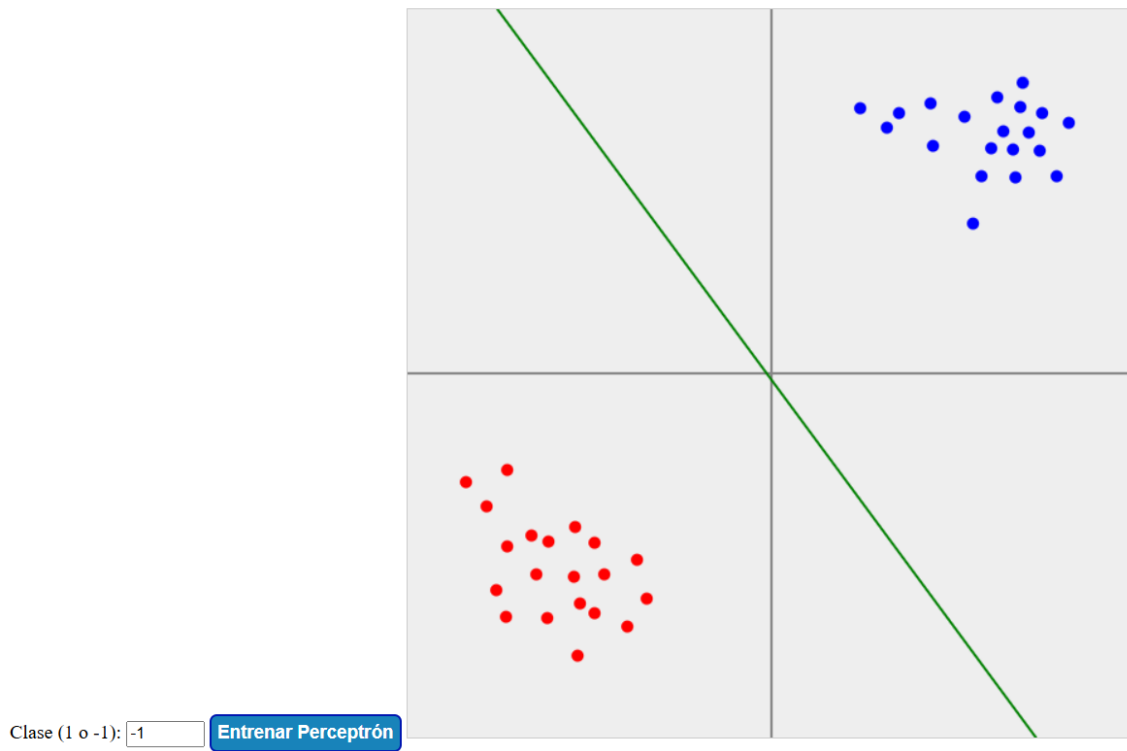
Perceptrón entrenando sobre puntos linealmente separables



Código del perceptrón modificado ya agregado al código del archivo:
"generar_conjunto de datos.html".

Perceptrón entrenando sobre puntos generados manualmente

Haz clic en el canvas para agregar puntos. Clase 1 = Azul, Clase -1 = Rojo



Actividad 7

Punto 1

El programa genera un total de 40 puntos. Los primeros 20 pertenecen a una clase (llamada clase 0) y están ubicados en la parte inferior izquierda del plano. Los otros 20 pertenecen a una segunda clase (clase 1) y están ubicados en la parte superior derecha. Estos datos se crean de forma que se puedan dividir perfectamente con una línea recta.

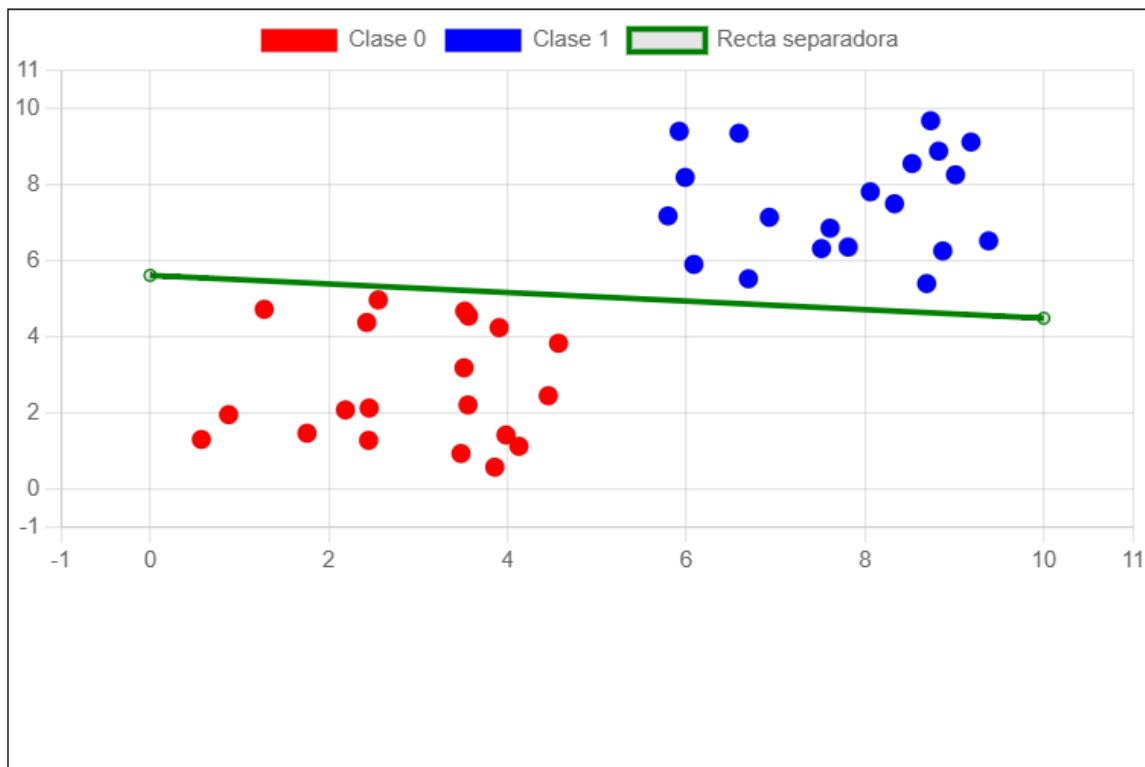
Después, se entrena un perceptrón. El perceptrón comienza con valores aleatorios y, tras repetir varias veces un proceso de corrección (en este caso, 100 veces), va ajustando sus parámetros para aprender a distinguir entre los dos grupos de puntos. Durante este entrenamiento, el perceptrón compara sus predicciones con las verdaderas clases y modifica sus valores internos para reducir los errores.

Una vez terminado el entrenamiento, el programa calcula la ecuación de la línea que el perceptrón ha aprendido como frontera de decisión entre las dos clases. Esta línea se obtiene a partir de los valores ajustados durante el entrenamiento.

Por último, la página muestra una gráfica que contiene:

- Los puntos de clase 0 en color rojo.
- Los puntos de clase 1 en color azul.
- La línea separadora en color verde.

Perceptrón con 40 Datos Linealmente Separables



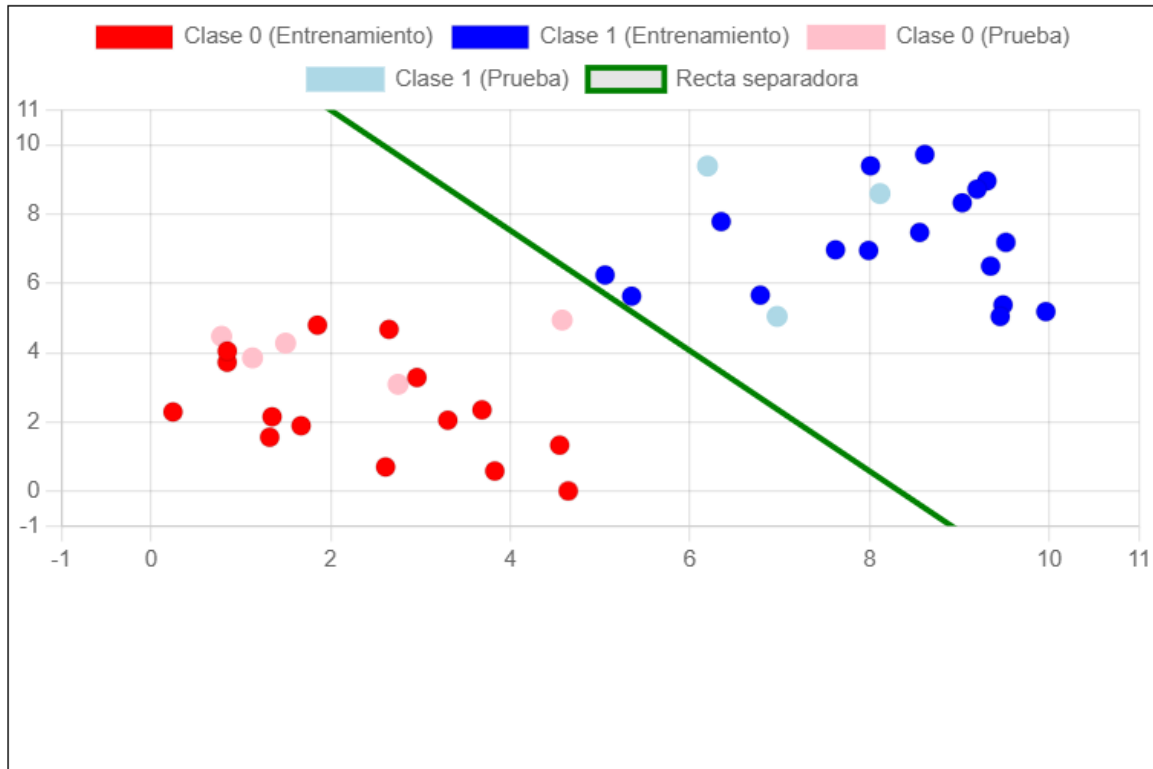
$$\text{Recta: } 0.05921624980910506x + 0.5252814561977609y + -2.9514766197894255 = 0$$

Punto 2

Primero, genera 40 puntos divididos en dos clases separables por una línea. Luego, los divide: el 80% se usa para entrenar al perceptrón y el 20% para probarlo. Tras el entrenamiento, se calcula la recta que separa ambas clases, y se muestra en una

gráfica junto con los puntos de entrenamiento (en colores fuertes) y de prueba (en colores suaves). También se muestra la ecuación de la recta aprendida.

Perceptrón con Conjunto de Entrenamiento y Prueba

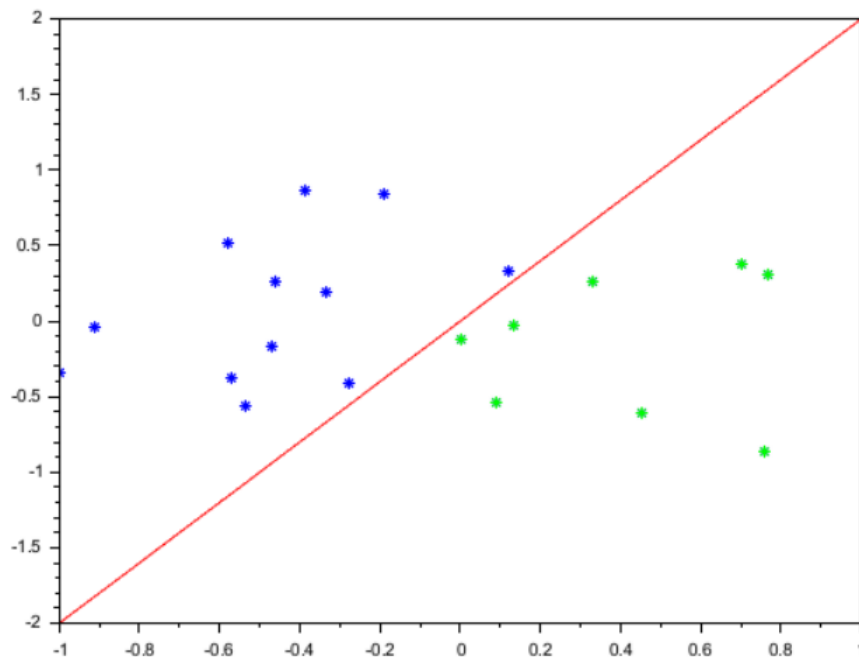


Recta: $0.42287333098893737x + 0.24367591418077517y + -3.5249703432253248 = 0$

Actividad 8

Punto 1

20 puntos clasificados generados



Punto 2

El programa comienza generando 20 puntos aleatorios dentro del cuadrado cuyos límites son de -1 a 1 tanto en el eje X como en el eje Y. Esto crea un conjunto de puntos dispersos en el plano.

Luego, el programa define una línea arbitraria de clasificación: $y = 2x$, que también se puede expresar como $y - 2x = 0$. Esta línea se utiliza como frontera para determinar de qué lado se encuentra cada punto: los que están por encima de la línea serán de una clase, y los que están por debajo, de otra.

Cada punto se compara con la línea para determinar su clase. Si el punto está por encima de la línea, se le asigna una clase positiva (+1), y si está por debajo, una clase negativa (-1). El programa también dibuja estos puntos en el gráfico, usando un color para cada clase.

Se define un perceptrón simple con pesos iniciales en cero. El perceptrón se utiliza para predecir la clase de un punto mediante una combinación lineal de sus

coordenadas (multiplicadas por los pesos). En un inicio, las predicciones del perceptrón no coinciden con las clases reales, ya que sus pesos aún no han sido ajustados.

Se aplica el algoritmo de entrenamiento del perceptrón. En cada iteración, se comparan las predicciones del perceptrón con las clases reales de los puntos. Si hay errores, se ajustan los pesos del perceptrón utilizando uno de los puntos mal clasificados. Este proceso se repite hasta que todos los puntos están correctamente clasificados o no hay errores restantes. El aprendizaje es supervisado y se realiza por corrección de errores.

Una vez que el perceptrón ha aprendido a clasificar correctamente los puntos, se dibuja la nueva línea de decisión aprendida en el gráfico. Esta línea puede coincidir o acercarse a la línea original usada para clasificar los puntos, dependiendo de la distribución de los datos.

Al final, el programa muestra en la consola los pesos finales que el perceptrón ha aprendido. Estos pesos definen la frontera de decisión y son el resultado del proceso de aprendizaje.

Actividad 9

Explicación de las diapositivas

La red neuronal construida trabaja con 4 entradas, que corresponden a las características de cada flor (como largo y ancho del sépalo y pétalo). La red cuenta con una única capa oculta compuesta por 10 neuronas, y la salida está formada por 3 neuronas, una por cada clase de flor: *Setosa*, *Versicolor* y *Virginica*. Se utiliza como función de activación la sigmoide, y el entrenamiento se realiza mediante el algoritmo de retropropagación.

Para entrenar el modelo, se usan 30 muestras del conjunto de datos Iris. Cada muestra tiene 4 características numéricas. Las clases de salida se representan con

el formato one-hot encoding, lo cual significa que cada tipo de flor se codifica como un vector binario: Setosa como [1, 0, 0], Versicolor como [0, 1, 0] y Virginica como [0, 0, 1].

El entrenamiento de la red neuronal se realiza con una tasa de aprendizaje de 0.1 y se repite durante 1000 iteraciones. Durante este proceso, se emplea la propagación hacia adelante para calcular las salidas de la red en cada paso, ajustando los pesos a medida que se minimiza el error.

Finalmente, en la fase de predicción, se aplicó el modelo entrenado a los datos de entrada. Se usó la función sigmoide para obtener las predicciones en forma de valores continuos, que luego se convirtieron a clases discretas. El desempeño del modelo se evaluó midiendo la precisión de la clasificación, es decir, qué tan frecuentemente el modelo acertó la clase correcta de flor.

Código de Scilab

El programa realiza el entrenamiento y evaluación de una red neuronal para clasificación, utilizando un conjunto de datos llamado "reprocessed.hungarian.data". Primero, se cargan los datos desde el archivo. Este archivo contiene una matriz donde cada fila representa un caso y cada columna una característica o la etiqueta de clase correspondiente.

Luego, se realiza una limpieza de los datos: se identifican los valores faltantes, que están representados por el valor -9. Para cada columna que contenga valores faltantes, se reemplazan estos por la media de los valores válidos de esa misma columna.

Después de la limpieza, se normalizan los datos para que todas las características (excepto la etiqueta de clase) estén dentro del mismo rango. Se utiliza una técnica llamada "normalización Min-Max", que transforma cada valor para que esté entre 0

y 1, lo que ayuda a mejorar la estabilidad y eficiencia del entrenamiento de la red neuronal.

Una vez normalizados los datos, se separan en dos grupos: uno con las características de entrada (las primeras 13 columnas) y otro con las etiquetas de salida (la columna 14).

Como las redes neuronales no pueden trabajar directamente con etiquetas categóricas, las etiquetas se convierten a formato "one-hot". Esto significa que si hay, por ejemplo, 4 clases posibles, cada etiqueta se convierte en un vector de longitud 4, donde solo una posición tiene el valor 1 y las demás son 0.

Luego, se define la arquitectura de la red neuronal. Esta red tendrá una capa de entrada con 13 neuronas, dos capas ocultas (una con 10 neuronas y otra con 5), y una capa de salida con tantas neuronas como clases tenga el problema. Se utiliza la función de activación logística (sigmoide) en todas las capas.

La red se entrena usando el algoritmo de retropropagación (backpropagation) durante 1000 iteraciones o "épocas", con una tasa de aprendizaje fija. Durante este proceso, la red ajusta sus pesos internos para minimizar el error entre sus predicciones y los valores reales.

Una vez finalizado el entrenamiento, la red se utiliza para predecir las salidas del mismo conjunto de datos. Las salidas de la red son vectores de probabilidades, y se selecciona como predicción la clase con la probabilidad más alta.

Finalmente, se compara cada predicción con su etiqueta real, y se calcula la precisión del modelo, es decir, el porcentaje de aciertos sobre el total de ejemplos.

Actividad 10

Estructura de la Neurona

La salida de la neurona se obtiene mediante la **función de activación sigmoide**, que es una función matemática utilizada para introducir no linealidad.

Algoritmo de Entrenamiento (Descenso de Gradiente)

El entrenamiento de la red neuronal se realiza utilizando el **algoritmo de retropropagación** (backpropagation). El proceso de entrenamiento involucra dos fases clave: **propagación hacia adelante** y **retropropagación del error**.

- **Propagación hacia adelante:** Las entradas de la red se pasan a través de las capas, calculando las activaciones hasta llegar a la capa de salida. Para cada capa, las salidas se calculan aplicando los pesos y la función de activación sigmoide.
- **Cálculo del error:** Una vez que se obtiene la salida de la red, se calcula el **error** comparando la salida predicha $\hat{Y}$ con la salida real Y . El error es la diferencia entre estos dos valores.
- **Retropropagación del error:** El error se propaga hacia atrás a través de la red, desde la capa de salida hasta la capa de entrada. Se utiliza la derivada de la función de activación para ajustar los pesos y reducir el error.

Cálculo del Gradiente

Durante el proceso de retropropagación, el gradiente de los pesos se calcula utilizando la regla de la cadena. La **derivada** de la función sigmoide es esencial para ajustar los pesos correctamente.

Arquitectura de la Red

La red tiene la siguiente estructura:

- **Entrada:** 4 características por ejemplo (4 entradas).
- **Capa oculta:** 10 neuronas en la capa oculta.
- **Salida:** 3 neuronas en la capa de salida.

Para el entrenamiento, se tienen **30 ejemplos de datos** (XX) y sus respectivas salidas correctas (YY).

Implementación en Scilab

En este código, se inicializan los pesos y sesgos de manera aleatoria y luego se realizan las siguientes operaciones:

- Propagación hacia adelante: Calcular las salidas de cada capa.
- Cálculo del error.
- Retropropagación: Calcular las derivadas de los pesos y actualizarlos.

Algunas funciones importantes de este código incluyen:

- **Sigmoid**: Función de activación sigmoide.
- **Sigmoid_derivada**: Derivada de la función sigmoide, que se utiliza durante la retropropagación.
- **Repmat**: Función para replicar una matriz, usada para expandir los sesgos b1b1 y b2b2 a las dimensiones correctas.

Detalles de Cálculos

Se proporciona una explicación detallada de cómo se calculan las **entradas ponderadas** en cada capa y cómo se aplican los sesgos. También se detalla cómo se calculan las **derivadas** necesarias para la retropropagación y cómo se **actualizan los pesos** para minimizar el error.

Actualización de Pesos y Sesgos

Una vez que se ha calculado el gradiente, los pesos se actualizan en función de la tasa de aprendizaje y el gradiente calculado. Este proceso se repite durante **varias épocas** hasta que el modelo converge a un mínimo local en la función de pérdida, lo que significa que la red ha aprendido a predecir correctamente.